

The Context Tree Weighting Algorithm

Chapter 4

Slides based on: Marcus Hutter, Elliot Catt, David Quarel
An Introduction to Universal Artificial Intelligence, CRC Press, 2024, Chapter 4

Outline

- 1 Introduction
- 2 4.1 Krichevsky–Trofimov (KT) Estimator
- 3 4.2 Context
- 4 4.3 Variable Length Context
- 5 4.4 Mixing Distributions
- 6 4.5 Context Tree Weighting
- 7 4.6 Exercises
- 8 4.7 History and References

Why This Chapter?

- In Chapter 3 we saw that Bayesian sequence prediction can learn the dynamics of an unknown environment, but computing a Bayes mixture over a huge model class can be prohibitively expensive.
- **Context Tree Weighting (CTW)** is a prediction algorithm with both strong theoretical guarantees *and* a genuinely practical, fast implementation.
- CTW computes a Bayesian mixture over *all* environments that depend on at most the previous D observations (a model class of size $O(2^{2^D})$), yet it can update its mixture weights in only $O(D)$ time per bit — a double-exponential speedup.
- Throughout this chapter, sequences are over the binary alphabet $\mathbb{B} = \{0, 1\}$ (most results generalize to any finite alphabet). We write 0 and 1 in typewriter font for symbols in a sequence, and 0, 1 in normal font for the natural numbers zero and one.

Roadmap

KT estimator (i.i.d. prediction) $\rightarrow$ fixed-length context (Markov) $\rightarrow$ variable-length context (suffix trees) $\rightarrow$ mixing distributions $\rightarrow$ Context Tree Weighting.

The KT Estimator: Definition I

Given a binary sequence $x_{1:t}$ generated by an unknown $\text{Bern}(\theta)$ process (where θ (theta) is the probability of sampling a 1), the probability of observing a specific sequence with a zeros and b ones is $\theta^b(1 - \theta)^a$. Using Jeffrey's prior $w(\theta) = \pi^{-1}[\theta(1 - \theta)]^{-1/2}$ and integrating out θ gives the **KT estimator**.

Definition 4.1.1 (KT estimator)

$$P_{\text{KT}}(x_{1:n}) \equiv P_{\text{KT}}(a, b) := \frac{1}{\pi} \int_0^1 \frac{1}{\sqrt{\theta(1 - \theta)}} \theta^b (1 - \theta)^a d\theta = \frac{1}{\pi} \text{Beta}\left(a + \frac{1}{2}, b + \frac{1}{2}\right)$$

where $\text{Beta}(a, b) := \int_0^1 \theta^{a-1} (1 - \theta)^{b-1} d\theta$ is the Beta function and a, b are simply the counts of zeros and ones in $x_{1:n}$ (the order of bits does not matter since we assumed i.i.d. sampling).

Plain-English meaning. a is how many 0s we have seen, b is how many 1s we have seen. $P_{\text{KT}}(a, b)$ is our best guess (after averaging over every possible value of the unknown bias θ , weighted by how plausible that θ was a priori) of how likely it was to see exactly this sequence of zeros and ones.

The KT Estimator: Definition II

Lemma 4.1.2 (Recursive KT estimator)

$$P_{\text{KT}}(0, 0) = 1, \quad P_{\text{KT}}(a + 1, b) = \frac{a + \frac{1}{2}}{a + b + 1} P_{\text{KT}}(a, b), \quad P_{\text{KT}}(a, b + 1) = \frac{b + \frac{1}{2}}{a + b + 1} P_{\text{KT}}(a, b)$$

Equivalently, writing the joint probability as a product of one-step predictions,

$$P_{\text{KT}}(x_{1:t}) = P_{\text{KT}}(x_t | x_{<t}) P_{\text{KT}}(x_{<t}), \quad P_{\text{KT}}(\epsilon) = 1,$$

$$P_{\text{KT}}(1 | x_{1:t}) = \frac{b + \frac{1}{2}}{a + b + 1}, \quad P_{\text{KT}}(0 | x_{1:t}) = 1 - P_{\text{KT}}(1 | x_{1:t}) = \frac{a + \frac{1}{2}}{a + b + 1}$$

Here ϵ (epsilon) denotes the empty sequence, $x_{<t}$ is shorthand for $x_1 x_2 \cdots x_{t-1}$ (everything observed before time t), and a, b are the counts of zeros and ones within $x_{<t}$. This recursion is exactly Laplace's rule of succession with a $+\frac{1}{2}$ (instead of $+1$) smoothing term, and it is what makes the KT estimator cheap to update: each new bit only requires one multiplication.

The KT Estimator: Definition III

Figure 4.1: Table of $P_{KT}(a, b)$

$a \backslash b$	0	1	2	3	4
0	1	$1/2$	$3/8$	$5/16$	$35/128$
1	$1/2$	$1/8$	$1/16$	$5/128$	$7/256$
2	$3/8$	$1/16$	$3/128$	$3/256$	$7/1024$
3	$5/16$	$5/128$	$3/256$	$5/1024$	$5/2048$

Note the symmetry $P_{KT}(a, b) = P_{KT}(b, a)$, visible in the table.

Example 4.1.3

Given the sequence 101111 we have $a = 1$ zero and $b = 5$ ones. The KT estimator predicts the next bit is a 1 with probability

$$P_{KT}(1 | 101111) = \frac{b + \frac{1}{2}}{a + b + 1} = \frac{5 + \frac{1}{2}}{1 + 5 + 1} = \frac{11}{14}.$$

The KT Estimator: Definition IV

Remark 4.1.4 (KT cannot see non-i.i.d. structure)

The KT estimator cannot distinguish the sequences 01010101... and 00110011... from a purely random $\text{Bern}(\frac{1}{2})$ sequence: it only counts zeros and ones and ignores order, so on these perfectly predictable patterns it performs no better than random guessing. Both example sequences are Markov (Definition 4.2.6), not i.i.d., and the KT estimator is only guaranteed to work well on i.i.d. data.

Lemma 4.1.5 (Lower bound)

For $a + b \geq 1$,

$$P_{\text{KT}}(a, b) \geq \frac{1}{2\sqrt{a+b}} \left(\frac{a}{a+b} \right)^a \left(\frac{b}{a+b} \right)^b.$$

The KT Estimator: Definition V

Theorem 4.1.6 (Instantaneous KT-convergence rate)

If $x_{1:\infty}$ is drawn from $\text{Bern}(\theta)$, then for all $\varepsilon > 0$ (epsilon > 0),

$$\mathbb{P}(|P_{\text{KT}}(x_{t+1} = 1 \mid x_{1:t}) - \theta| \geq \varepsilon) \leq 15 e^{-2\varepsilon^2 t}.$$

In words: the chance that the KT estimator's prediction is off by more than ε shrinks exponentially fast as we see more data ($t \rightarrow \infty$), so the KT estimator learns the true bias θ of a coin very quickly.

Proof idea. Write $P_{\text{KT}}(x_{t+1} = 1 \mid x_{1:t}) = \frac{b_t + \frac{1}{2}}{t+1}$ where b_t is the number of ones seen. This differs from the raw empirical frequency b_t/t by at most $\frac{1}{2t}$, and Hoeffding's inequality (Theorem 2.2.61) bounds how far the empirical frequency itself can be from θ .

Redundancy: Measuring Coding/Prediction Performance

When the true distribution μ (mu) is unknown, we approximate it by a *coding distribution* P_c . By Shannon's source coding theorem, encoding a sequence $x_{1:n}$ with an (assumed) distribution $\hat{\rho}$ costs on average $-\log_2 \hat{\rho}(x_{1:n})$ bits. The extra cost of using an imperfect predictor is measured by *redundancy*.

Definition 4.1.10 (Redundancy)

For distributions $\rho, \hat{\rho}$ and sequence $x_{1:n}$, the *individual cumulative redundancy* of $\hat{\rho}$ with respect to ρ is

$$r_{\hat{\rho}, \rho}(x_{1:n}) := \log_2 \frac{1}{\hat{\rho}(x_{1:n})} - \log_2 \frac{1}{\rho(x_{1:n})} \equiv \log_2 \rho(x_{1:n}) - \log_2 \hat{\rho}(x_{1:n}).$$

When $\rho = \mu$ (the true distribution) we simply write $r_{\hat{\rho}}(x_{1:n})$.

Plain-English meaning: redundancy is the number of *extra* bits (beyond the theoretical minimum) that we waste per sequence when we encode using our approximate predictor $\hat{\rho}$ instead of the true ρ .

Lemma 4.1.11 (KT estimator redundancy)

For all $a + b \geq 1$ and all $\theta \in [0, 1]$,

$$r_{\text{KT},\theta}(a, b) = \log_2 \frac{(1 - \theta)^a \theta^b}{P_{\text{KT}}(a, b)} \leq \frac{1}{2} \log_2(a + b) + 1.$$

Unlike Theorem 4.1.6 (which needs i.i.d. data), this bound holds for *any* sequence and grows only logarithmically with the amount of data seen — a very strong guarantee.

Python: The KT Estimator in Practice

```
from math import lgamma, exp, pi

def P_KT(a, b):
    """Block probability P_KT(x_1:n) given a zeros, b ones."""
    return exp(lgamma(a + 0.5) + lgamma(b + 0.5) - lgamma(a + b + 1)) / pi

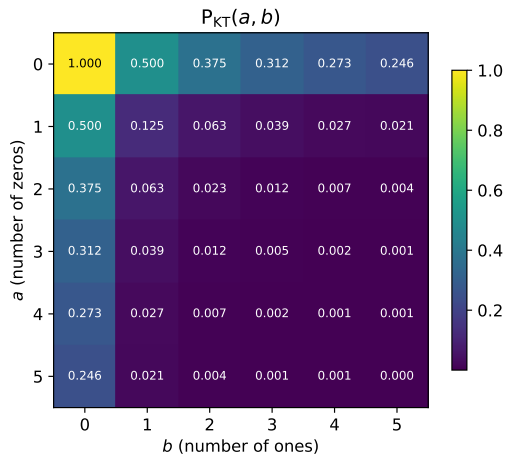
def kt_predict_one(a, b):
    """P_KT(next bit = 1 | a zeros, b ones seen so far)."""
    return (b + 0.5) / (a + b + 1)

# Example 4.1.3: sequence "101111" -> a=1 zero, b=5 ones
a, b = 1, 5
print(kt_predict_one(a, b))          # 0.7857142857142857 == 11/14

# Running the recursive KT estimator bit-by-bit on 101111
a = b = 0
seq = [1, 0, 1, 1, 1, 1]
for x in seq:
    p1 = kt_predict_one(a, b)
    print(f"context so far has (a={a},b={b}) -> P(next=1) = {p1:.4f}")
    if x == 1:
        b += 1
    else:
        a += 1
```

Visualizing $P_{KT}(a, b)$

The heat-map below shows $P_{KT}(a, b)$ for $a, b \in \{0, \dots, 5\}$ (Figure 4.1 generalised): darker/lower values occur when the counts are large and roughly balanced, since a highly mixed sequence of zeros and ones is intrinsically less predictable (and hence has a lower block probability) than a very lopsided one.



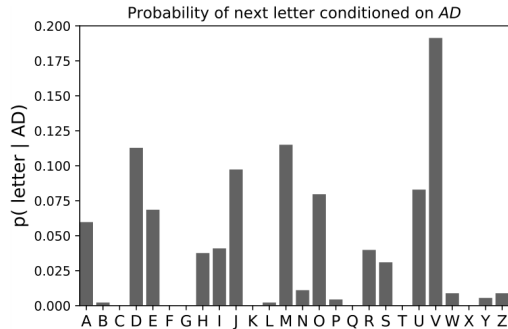
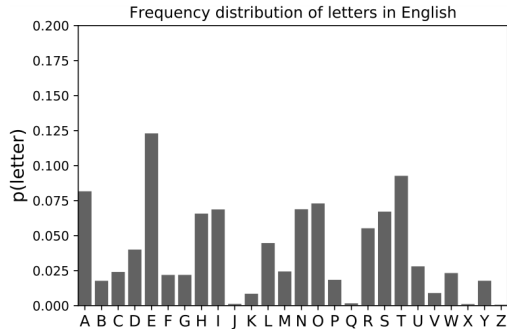
Why Context Matters

The KT estimator only uses the *frequency* of 0s and 1s; it ignores the *order* in which they appear. But for real sequences, recent history often carries the most useful information — this recent information is called the **context**.

Example 4.2.1 (Context in language)

Given the letters AD and asked to guess the next letter of an English word, using no context at all we would guess E (the most frequent letter overall) — a poor choice, since almost no words start with ADE or ADT. Conditioning on the context AD, a good predictor assigns high probability to V and J (words like *advance*, *adjust*), which the context-free frequency estimate would never have suggested.

Context Reshapes the Distribution



Left: unconditional letter frequency in English. Right: probability of the next letter given the context AD
— using context reshapes the distribution entirely.

Markov Distributions

Definition 4.2.3 (Markov)

A probability distribution $P : \mathbb{B}^* \rightarrow \Delta\mathbb{B}$ is *Markov* if $P(x_t | x_{<t}) = P(x_t | x_{t-1})$ for all t : the distribution of the next symbol depends *only* on the single most recently observed symbol (and not on the time step t itself).

Plain-English meaning: once you know the very last bit, no earlier history helps you predict the next bit any better. Every i.i.d. process is trivially Markov (it ignores context entirely), so Markov is a strictly more general/powerful notion than i.i.d.

Example 4.2.2 (KT fails on non-i.i.d. data)

For the alternating sequence $x_{1:2t} = (01)^t$, the KT estimator gives $P_{\text{KT}}(0 | x_{1:2n}) = \frac{n+\frac{1}{2}}{2n+1} = \frac{1}{2}$ — no better than a coin flip — even though the pattern is perfectly predictable once you look at just the last bit.

Remedy: instead of counting single symbols, count *digraphs* — how often each symbol is followed by 0 or 1 — and run a separate KT estimator for each possible “most recent symbol.”

k-Markov Environments

Definition 4.2.6 (*k*-Markov)

$P : \mathbb{B}^* \rightarrow \Delta\mathbb{B}$ is *k*-Markov (a k^{th} -order Markov model) if $P(x_t | x_{<t}) = P(x_t | x_{t-k:t-1})$ for all t : the next symbol depends only on the k most recently observed symbols.

We define the *KT counts* of a string x : $a_s(x)$ is the number of occurrences of substring $s0$ in x , and $b_s(x)$ the number of occurrences of $s1$. Then we run a KT estimator per context s :

$$P_k(0 | x_{1:n}) = \frac{a_s + \frac{1}{2}}{a_s + b_s + 1}, \quad P_k(1 | x_{1:n}) = \frac{b_s + \frac{1}{2}}{a_s + b_s + 1} = 1 - P_k(0 | x_{1:n})$$

where $s = x_{n-k+1:n}$ is the most recent k bits (**the context**).

Example 4.2.5 (1-Markov KT)

On the sequence $x_{1:6} = 010101$, since the most recent bit is 1, we count digraphs 10 (twice) and 11 (zero times), giving $P_1(x_7 = 0 | x_{1:6}) = \frac{2+\frac{1}{2}}{2+0+1} = \frac{5}{6}$: the 1-Markov estimator is confidently correct, unlike the plain KT estimator.

Choosing k : Too Small, Too Large

Example 4.2.8/2.9: $k = 0, 1, 2$ on $(100)^n$

On the sequence $(100)^n$ (a 2-Markov source), the k -Markov estimators give

$$P_0(0 \mid (100)^n) \rightarrow \frac{2}{3}, \quad P_1(0 \mid (100)^n) \rightarrow \frac{1}{2}, \quad P_2(0 \mid (100)^n) \rightarrow 0.$$

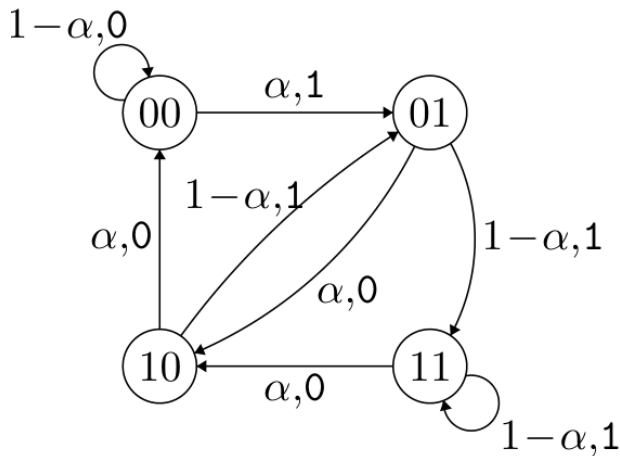
Only $k = 2$ correctly learns the deterministic rule $00 \rightarrow 1$, $01 \rightarrow 0$, $10 \rightarrow 0$. Surprisingly $k = 1$ is *worse* than $k = 0$: with only one bit of context, digraphs 01 and 00 occur about equally often, so the 1-Markov model cannot tell them apart and gains nothing over the naive (i.i.d.) estimate.

Remark 4.2.12 (Problem of k -Markov for large k)

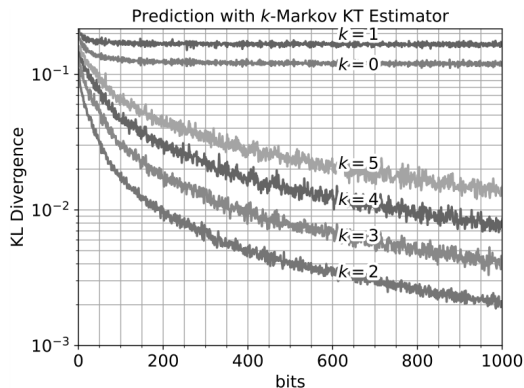
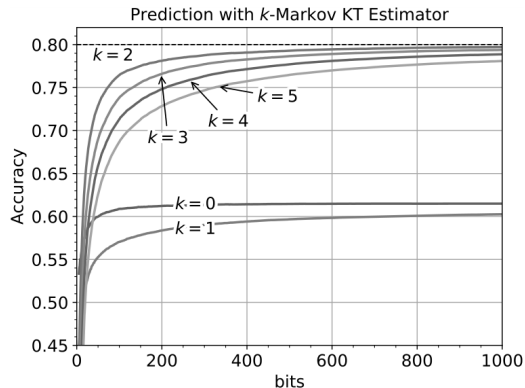
A k -Markov estimator has 2^k parameters to learn, so the sample size *per parameter* is only $O(n/2^k)$. Too small a k means the model can never learn the true pattern; too large a k wastes data on irrelevant context and slows convergence. If the environment truly is k -Markov but k is unknown, we need a way to *learn* the right amount of context — this motivates variable-length context in Section 4.3.

k-Markov Experiments: A Noisy 2-Markov Source

Consider the 2-Markov chain μ_1 below, parameterized by α (alpha) $\in [0, 1]$: with probability α the “clean” bit of the pattern $(100)^n$ is sampled, and with probability $1 - \alpha$ a noisy/wrong bit is sampled instead (and the context updates accordingly). Each state is labeled by its 2-bit context; edges show (sampling probability, sampled bit).



k -Markov Experiments: Accuracy and KL Divergence



Left: prediction accuracy vs. number of bits seen, for k -Markov KT estimators, $\alpha = 0.8$. Right: instantaneous KL divergence (Eq. 4.2.11) between the true μ_1 and each estimator's prediction — lower is better. Any $k \geq 2$ eventually learns the pattern, but larger k (more parameters) learns more slowly.

Python: k -Markov KT Estimator

```
def run_kmarkov_kt(bits, k):
    """Simulate a  $k$ -Markov KT estimator on a bit sequence, return per-step  $P(\text{next}=1)$ ."""
    counts = {}    # context (tuple of  $k$  bits) -> [a, b]
    preds = []
    ctx = tuple([0] * k) # pad with zeros before enough context accumulates
    for t, x in enumerate(bits):
        a, b = counts.get(ctx, [0, 0])
        p1 = (b + 0.5) / (a + b + 1)
        preds.append(p1)
        # update counts for the context we just used
        if x == 1:
            b += 1
        else:
            a += 1
        counts[ctx] = [a, b]
        # slide the context window forward by one bit
        ctx = (ctx + (x,))[-k:] if k > 0 else ()
    return preds

# demonstrate  $k=0$  vs  $k=2$  on  $(100)^n$ 
bits = [1, 0, 0] * 40
p0 = run_kmarkov_kt(bits, k=0)
p2 = run_kmarkov_kt(bits, k=2)
print("k=0 final  $P(\text{next}=1)$ :", round(p0[-1], 3))    # -> ~0.33 (matches global 1/3 frequency)
print("k=2 final  $P(\text{next}=1)$ :", round(p2[-1], 3))    # -> ~0.0 or ~1.0 depending on context
```

Motivation: Not All Context Needs the Same Length

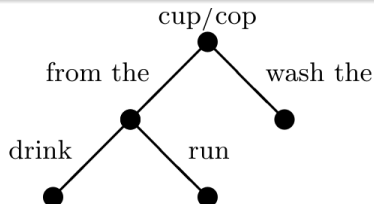
A k -Markov estimator *always* uses exactly k bits of context, even when fewer would do. Recall the optimal predictor for μ_1 needs the rules $00 \rightarrow 1$, $01 \rightarrow 0$, $10 \rightarrow 0$, $11 \rightarrow 0$ — but this simplifies to just

$$00 \rightarrow 1 \quad 10 \rightarrow 0 \quad 1 \rightarrow 0$$

If the last bit was 1, one bit of context suffices; only if the last bit was 0 do we need to look at the second-to-last bit too. We want predictors that can use *variable-length* context — long when needed, short when not.

Example 4.3.1 (Context tree, in language)

Given context from the, we may need to look further back (run from the vs. drink from the) to decide cop vs. cup. But given context wash the, no further context is needed — cup is almost certain. This asymmetric context requirement is naturally represented as a tree:



Suffixes and Suffix Sets

Definition 4.3.2 (Suffix)

A string x is a *suffix* of string y if there exists a string z with $y = zx$.

E.g., 10 is a suffix of 0010.

Definition 4.3.3 (Suffix set)

A *suffix set* $\mathcal{S}$ is a finite set of binary strings that is *complete* and *proper*: it is *proper* if no member of $\mathcal{S}$ is a suffix of another member, and *complete* if every semi-infinite (leftward) sequence $\dots x_{t-1}x_t$ has a suffix belonging to $\mathcal{S}$.

Plain-English meaning: a suffix set is a collection of “recent-history patterns” that (a) never overlap in a redundant way (properness) and (b) always has *some* pattern that matches any actual history (completeness) — so every possible context maps to exactly one entry of $\mathcal{S}$.

Uniqueness of Suffixes

Proposition 4.3.4 (Uniqueness of suffix)

Given a suffix set $\mathcal{S}$, every semi-infinite sequence ω (omega) has a *unique* suffix in $\mathcal{S}$.

Example 4.3.5

$\mathcal{S} = \{1, 11, 101, 001\}$ is proper and complete.

Suffix Trees

Definition 4.3.6 (Suffix tree)

A *suffix tree* is a finite binary tree, defined recursively as

$$\text{Tree} = \text{Leaf} \mid \text{Node Tree Tree}$$

Every suffix set $\mathcal{S}$ corresponds to a unique suffix tree $\Psi_{\mathcal{S}}$: start at a root node, and for each string $s \in \mathcal{S}$, read s **in reverse (from right to left)**, walking down the tree — a 0 adds/walks to a **right** child, a 1 adds/walks to a **left** child. Reading in reverse reflects that the most recent bit x_n (the last one) should be examined *first* when tracing context.

Definition 4.3.7 (Model class $\mathcal{C}_D$)

The model class of depth D is the set of all complete, proper suffix sets with strings of length at most D :

$$\mathcal{C}_D := \begin{cases} \{\{\epsilon\}\} & D = 0 \\ \{\{\epsilon\}\} \cup \{\mathcal{S}_1\{0\} \cup \mathcal{S}_2\{1\} : \mathcal{S}_1, \mathcal{S}_2 \in \mathcal{C}_{D-1}\} & D > 0 \end{cases}$$

where $AB := \{ab : a \in A, b \in B\}$ denotes concatenation of string sets.

The Model Class $\mathcal{C}_D$: What It Means

Plain-English meaning: $\mathcal{C}_D$ is simply the set of *every* suffix set that never needs more than D bits of context — this is our model class, the collection of all environments we are willing to consider.

The Model Class $\mathcal{C}_D$ Grows Fast

Example 4.3.8

$$\mathcal{C}_0 = \{\{\epsilon\}\}, \quad \mathcal{C}_1 = \{\{\epsilon\}, \{0, 1\}\}, \quad \mathcal{C}_2 = \{\{\epsilon\}, \{0, 1\}, \{00, 10, 1\}, \{0, 01, 11\}, \{00, 01, 10, 11\}\}$$

Proposition 4.3.9 (Model class inclusions)

$\mathcal{C}_0 \subseteq \mathcal{C}_1 \subseteq \mathcal{C}_2 \subseteq \dots$ — a deeper model class contains every shallower one as a special case.

Lemma 4.3.10 (Double-exponential size)

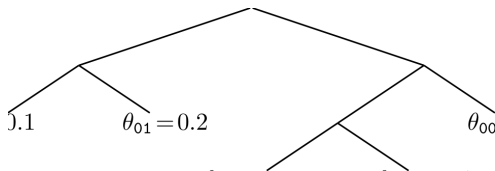
For $D \geq 1$: $2^{2^{D-1}} \leq |\mathcal{C}_D| \leq 2^{2^D}$.

This is the key difficulty CTW must overcome: even $D = 5$ gives a model class with roughly $2^{32} \approx 4$ billion possible environments — far too many to enumerate one by one, yet CTW will handle it in time *linear* in D .

Definition 4.3.11 (Prediction Suffix Tree, PST)

A PST $\Psi_{\mathcal{S}, \Theta_{\mathcal{S}}}$ is a suffix tree whose leaves each carry a parameter $\theta_s \in [0, 1]$ (the probability the next bit is 1 given context s):
PST = Leaf θ | Node PST PST.

Predicting with a PST



Example 4.3.14

For $\mathcal{S} = \{11, 01, 110, 010, 00\}$ with parameters $\theta_{11}=0.1, \theta_{01}=0.2, \theta_{110}=0.3, \theta_{010}=0.4, \theta_{00}=0.5$: given the sequence 0101101, we compute $\beta_{\mathcal{S}}(0101101) = 01$ (its suffix in $\mathcal{S}$), so

$$P_{\mathcal{S}, \Theta_{\mathcal{S}}}(x_t = 1 \mid x_{<t} = 0101101) = \theta_{01} = 0.2.$$

Definition 4.3.12 (Suffix function $\beta_{\mathcal{S}}$)

$\beta_{\mathcal{S}} : \mathbb{B}^{\infty} \rightarrow \mathbb{B}^*$ maps a semi-infinite sequence onto its unique suffix in $\mathcal{S}$. $\beta_{\mathcal{S}}$ depends only on the last $D := \max_{s \in \mathcal{S}} \ell(s)$ bits of the input (padding with zeros if fewer than D bits are available).

PST Probability as a Product

Lemma 4.3.16 (PST probability as a product)

$$P_{\mathcal{S}, \Theta_{\mathcal{S}}}(x_{1:n}) = \prod_{s \in \mathcal{S}} \theta_s^{b_s} (1 - \theta_s)^{a_s}$$

where a_s, b_s are the KT counts of $x_{1:n}$: the block probability factors into one term per context in the suffix set, exactly as if we ran $|\mathcal{S}|$ independent i.i.d. processes.

Algorithm 4.1: Building a PST from $(\mathcal{S}, \Theta_{\mathcal{S}})$

Algorithm 4.1 (Initializing PST from $\mathcal{S}$ and $\Theta_{\mathcal{S}}$)

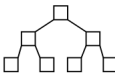
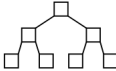
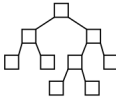
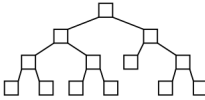
- 1 Initialize Ψ as an empty binary tree.
- 2 For each $s \in \mathcal{S}$: set $p := \Psi$ (start at root), and read s **right-to-left**. For each bit: if it is 1, add/walk to the **left** child of p ; if it is 0, add/walk to the **right** child.
- 3 Once s is fully consumed, store θ_s in the current leaf node p .

This is exactly how Figure 4.7's tree was built from $\mathcal{S} = \{11, 01, 110, 010, 00\}$.

Encoding Suffix Sets: Definition 4.3.17 ($\mathcal{E}_D$)

Given a fixed maximal depth D , a suffix tree $\Psi_{\mathcal{S}}$ is encoded as a bit string: ϵ if $D = 0$; 0 if $\Psi_{\mathcal{S}}$ is a leaf; otherwise 1 followed by the encodings of the left and right subtrees (at depth $D - 1$). Because the decoder knows the maximal depth D , this code is *uniquely decodable* (Theorem 4.3.19) even though some trees get surprisingly short codes.

Encoding Examples for Depths $D = 1, 2, 3$

$D = 1$	$\mathcal{S}$	$\{\varepsilon\}$	$\{1, 0\}$			
	$\Psi_{\mathcal{S}}$					
	$\mathcal{E}(\mathcal{S})$	0	1			
$D = 2$	$\mathcal{S}$	$\{\varepsilon\}$	$\{1, 0\}$	$\{1, 10, 00\}$	$\{11, 01, 0\}$	$\{11, 10, 01, 00\} =: \mathcal{S}_2$
	$\Psi_{\mathcal{S}}$					
	$\mathcal{E}(\mathcal{S})$	0	100	101	110	111
$D = 3$	$\mathcal{S}$	$\{\varepsilon\}$	$\{1, 0\}$	$\{1, 10, 00\}$	$\{11, 01, 0\}$	$\{11, 10, 01, 00\}$
	$\Psi_{\mathcal{S}}$					
	$\mathcal{E}(\mathcal{S})$	0	100	10100	11000	1100100
$D = 3$	$\mathcal{S}$	$\{1, 10, 100, 000\} =: \mathcal{S}_1$	$\{11, 01, 110, 010, 00\}$	$\{111, 011, 101, 011, 10, 100, 000\} =: \mathcal{S}_3$		
	$\Psi_{\mathcal{S}}$					
	$\mathcal{E}(\mathcal{S})$	10101	1100110	1111101		

Model Cost: How Complex is a Suffix Set?

Definition 4.3.20 (Model cost)

$$\Gamma_D(\mathcal{S}) := |\mathcal{S}| - 1 + |\{s \in \mathcal{S} : \ell(s) < D\}|$$

Plain-English meaning: the model cost is the number of leaves in the suffix tree, plus a penalty of 1 for every leaf that is *not* at the maximal depth D (i.e. every internal “non-leaf” node needs an extra bit to signal that it is not a leaf).

Theorem 4.3.22 (Model cost = encoding length)

$\Gamma_D(\mathcal{S}) = \ell(\mathcal{E}_D(\mathcal{S}))$: the model cost exactly equals the number of bits used by the encoding $\mathcal{E}_D$.

Example 4.3.23

For $\mathcal{S}_1 = \{1, 10, 100, 000\}$ (deep, narrow tree) and $\mathcal{S}_2 = \{11, 10, 01, 00\}$ (shallow, balanced tree), both with 4 elements: $\Gamma_3(\mathcal{S}_1) = 5$ but $\Gamma_2(\mathcal{S}_2) = 3$. **Balanced trees are cheaper** — the model cost rewards simplicity, which is exactly what we want for a Bayesian prior that avoids overfitting to spurious deep context.

Algorithms 4.2 & 4.3: PST Prediction and Update

Algorithm 4.2 (Suffix Tree Prediction)

Input: PST $\Psi_{\mathcal{S}, \Theta_{\mathcal{S}}}$, context $x_{1:t}$. **Output:** $\theta_{\beta_{\mathcal{S}}(x_{1:t})}$

- 1 For $i = t$ down to 1: if Ψ is a leaf, return $\Psi.\theta$; else walk to $\Psi.\text{left}$ if $x_i = 1$, else $\Psi.\text{right}$.

Algorithm 4.3 (Suffix Tree Update)

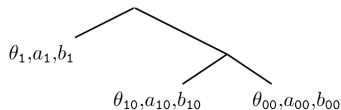
Input: PST updated on $x_{1:t}$; next bit $x = x_{t+1}$

- 1 Walk down the tree following $x_{1:t}$ (as in Algorithm 4.2) until reaching a leaf s .
- 2 If $x = 1$: $\Psi.b \ += 1$; else $\Psi.a \ += 1$.
- 3 Update $\Psi.\theta := (\Psi.b + \frac{1}{2}) / (\Psi.a + \Psi.b + 1)$ — this is exactly the recursive KT-estimator update (Lemma 4.1.2) applied at that one leaf.

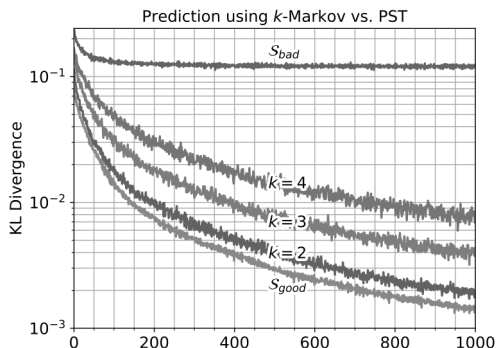
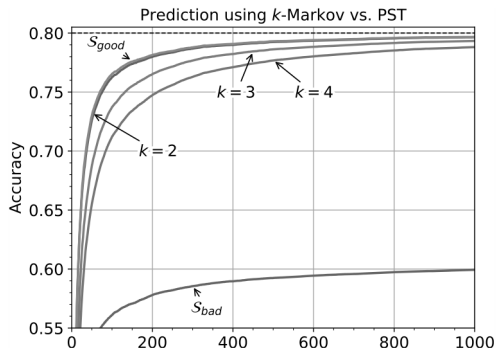
PST-KT: instead of fixing $\Theta_{\mathcal{S}}$ in advance, we *learn* it on-line using this update rule, giving the PST-KT probability $P_{\mathcal{S}, \text{KT}}(x_{1:n}) = \prod_{s \in \mathcal{S}} P_{\text{KT}}(a_s, b_s)$ (Definition 4.3.24) — one independent KT estimator running at each leaf of the tree.

PST Experiments: Known Structure Beats Fixed- k

Comparing the PST predictor with known suffix set $\mathcal{S}_{\text{good}} = \{1, 10, 00\}$ (Figure 4.9, learned via Algorithm 4.3) against fixed k -Markov KT estimators on the same noisy 2-Markov source μ_1 :



Because $\mathcal{S}_{\text{good}}$ already knows exactly which contexts matter (it merges the redundant contexts 10 and 00 into the leaf 1), it has fewer parameters to learn than $k = 2$ or $k = 3$, and hence learns *faster* while being just as accurate asymptotically.



Redundancy of Mixing Two Distributions

Example 4.4.1

Suppose $\hat{\mu}_1$ predicts μ_1 well and $\hat{\mu}_2$ predicts μ_2 well. Consider the simple average mixture $\hat{\mu}_w(x_{1:t}) := \frac{1}{2}\hat{\mu}_1(x_{1:t}) + \frac{1}{2}\hat{\mu}_2(x_{1:t})$. Then for either $i \in \{1, 2\}$:

$$r_{\hat{\mu}_w, \mu_i}(x_{1:n}) = \log_2 \frac{2}{\hat{\mu}_1(x_{1:t}) + \hat{\mu}_2(x_{1:t})} - \log_2 \frac{1}{\mu_i(x_{1:t})} \leq 1 + r_{\hat{\mu}_i, \mu_i}.$$

Plain-English meaning: mixing two good predictors together costs at most *one extra bit* of redundancy compared to using the best one alone — yet we get a single predictor that works well *no matter which* of the two environments is the true one. This is the core insight behind Bayesian mixtures: hedging your bets across many models is nearly free if you already have a good model for each possibility, and the same argument extends to an uneven mixture $\hat{\mu}_{w,\alpha} = \alpha\hat{\mu}_1 + (1-\alpha)\hat{\mu}_2$, or a mixture of arbitrarily many distributions with weights $\theta_i \geq 0, \sum_i \theta_i = 1$.

Since $\mathcal{C}_D$ can have up to 2^{2^D} suffix sets, we cannot afford one extra bit *per suffix set* in a naive mixture — we need a way to mix over *all* of $\mathcal{C}_D$ at once, efficiently. That is exactly what CTW does.

Setup

We now assume only that the true environment μ is *some* $P_{\mathcal{S}, \Theta_{\mathcal{S}}}$ for an *unknown* suffix set $\mathcal{S}$ and unknown parameters $\Theta_{\mathcal{S}}$, with the (weak) assumption that we know an upper bound D on how much context could ever be needed (so $\mathcal{S} \in \mathcal{C}_D$).

The natural Bayesian approach: fix the model class $\mathcal{C}_D$, put a prior weight $w_{\mathcal{S}}$ on each suffix set, and use the Bayes mixture (recall Section 3.1):

$$\xi(x_{1:n}) := \sum_{\mathcal{S} \in \mathcal{C}_D} w_{\mathcal{S}} P_{\mathcal{S}, \Theta_{\mathcal{S}}}(x_{1:n})$$

Naively this costs $O(2^{2^D})$ time — utterly infeasible even for modest D . **CTW** computes exactly this mixture (with weights $w_{\mathcal{S}} := 2^{-\Gamma_D(\mathcal{S})}$) in only $O(n \cdot D)$ time.

4.5.1 The CTW Algorithm I

Definition 4.5.1 (Context tree)

Given depth D , a *context tree* $\mathcal{T}_D$ is a *perfect* binary tree where every node s (path of length $\ell(s) \leq D$ from the root) stores KT counts (a_s, b_s) , the KT probability $P_{\text{KT}}(a_s, b_s)$, and a *weighted probability* $P_w(s)$ defined recursively:

$$P_w(s) := \begin{cases} \frac{1}{2}P_{\text{KT}}(a_s, b_s) + \frac{1}{2}P_w(0s)P_w(1s) & \ell(s) < D \\ P_{\text{KT}}(a_s, b_s) & \ell(s) = D \end{cases}$$

where $1s$ and $0s$ are the left/right children (context with a 1 or 0 prepended), and $a_s = a_{1s} + a_{0s}$, $b_s = b_{1s} + b_{0s}$.

Plain-English meaning: at every leaf ($\ell(s) = D$), we simply run the ordinary KT estimator, exactly as if the sequence following context s were i.i.d. At every internal node, we *mix* two hypotheses with equal weight $\frac{1}{2}$ each: either the KT estimator here is already good enough (first term), or we should look one bit deeper into the context and combine the (already-computed) weighted probabilities of both children (second term). This mirrors exactly the redundancy argument from Example 4.4.1.

4.5.1 The CTW Algorithm II

Algorithm 4.4 (CTW Update, conceptual)

For each new bit x_{t+1} , using its D -bit context $x_{t-D+1:t}$:

- 1 Walk from the root to the leaf s that matches the context, and at every node s' along the way update $a_{s'}$, $b_{s'}$ and $P_{\text{KT}}(a_{s'}, b_{s'})$ (Lemma 4.1.2).
- 2 Walk back up from the leaf to the root, and at each node recompute $P_w(s') = \frac{1}{2}P_{\text{KT}}(a_{s'}, b_{s'}) + \frac{1}{2}P_w(0s')P_w(1s')$.

Each update touches only the D nodes on the path from root to leaf, so the whole update costs $O(D)$ time — regardless of how large C_D is!

4.5.1 The CTW Algorithm III

Definition 4.5.4 (CTW probability)

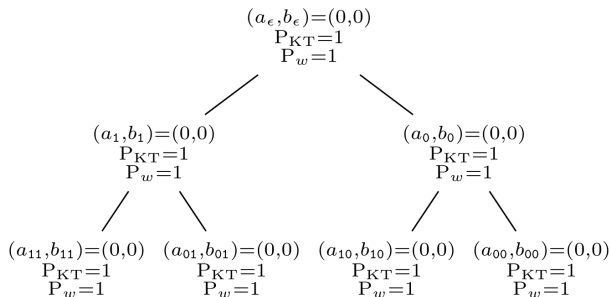
$P_D^{\text{CTW}}(x_{1:n}) := P_w(\epsilon)$, the weighted probability at the root after processing $x_{1:n}$. The conditional (predictive) probability is recovered as

$$P_D^{\text{CTW}}(1 | x_{<n}) = \frac{P_D^{\text{CTW}}(x_{<n}1)}{P_D^{\text{CTW}}(x_{<n})}.$$

4.5.1 The CTW Algorithm IV

Example 4.5.6 (Walking through a $D = 2$ update)

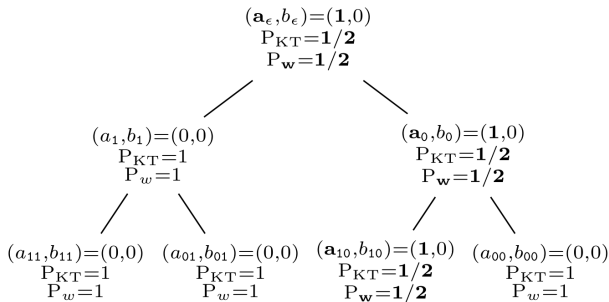
Start with a freshly initialized tree $\mathcal{T}_2$ (all counts zero, all $P_{KT} = P_w = 1$):



(a) A newly initialized context tree $\mathcal{T}_2$ of depth $D=2$.

4.5.1 The CTW Algorithm V

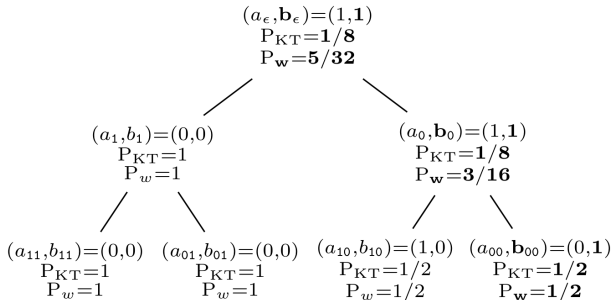
Feed in $x_{1:4} = 1001$. After observing $x_3 = 0$ given context $x_{1:2} = 10$ (the path $\epsilon \rightarrow 0 \rightarrow 10$ is updated, highlighted in black):



(b) The context tree $\mathcal{T}_2$ after having processed bit $x_3 = 0$ given context $x_{1:2} = 10$.

4.5.1 The CTW Algorithm VI

Then after observing $x_4 = 1$ given context $x_{1:3} = 100$ (path $\epsilon \rightarrow 0 \rightarrow 00$ updated):



(c) The context tree $\mathcal{T}_2$ after having processed bit $x_4 = 1$ given context $x_{1:3} = 100$.

Notice how $P_w(\epsilon) = 5/32$ at the root combines information from *both* updates — this single number already represents the Bayes mixture over *all five* suffix sets in $\mathcal{C}_2$ (Remark 4.5.7).

Unrolling $P_w(\epsilon)$ for Small D

Remark 4.5.7 (CTW is a mixture over all PSTs)

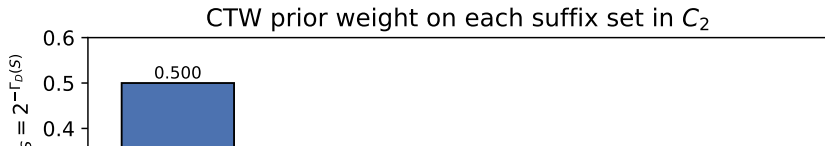
For $D = 0$: $P_w(\epsilon) = P_{\text{KT}}(a, b)$ (plain KT). For $D = 1$, writing $k(s) := P_{\text{KT}}(a_s, b_s)$:

$$P_w(\epsilon) = \frac{1}{2}k(\epsilon) + \frac{1}{2}k(0)k(1)$$

For $D = 2$, expanding fully:

$$P_w(\epsilon) = \frac{1}{2}\square + \frac{1}{8}\left(\wedge + \wedge^{\wedge} + \wedge^{\wedge} + \wedge^{\wedge}\right)$$

where each symbol corresponds to one of the five trees in $\mathcal{C}_2$: the empty tree $\square$ gets weight 2^{-1} (model cost 1), and each of the other four (depth-2) trees gets weight 2^{-3} (model cost 3) — matching Figure 4.8's encoding lengths exactly!



4.5.2 CTW Properties: The Main Theorem

Theorem 4.5.8 (Context Tree Weighting)

For any node $s \in \mathcal{T}_D$ with $d = \ell(s)$, and counts taken w.r.t. $x_{1:n}$:

$$P_w(s) = \sum_{S \in \mathcal{C}_{D-d}} 2^{-\Gamma_{D-d}(S)} \prod_{s' \in S} P_{\text{KT}}(a_{s's}, b_{s's})$$

Proof idea: by reverse induction on depth, using the recursive model-cost identity $\Gamma_{D+1}(\mathcal{V}\{0\} \cup \mathcal{W}\{1\}) = \Gamma_D(\mathcal{V}) + \Gamma_D(\mathcal{W}) + 1$ (Lemma 4.3.21) to show that splitting $P_w(s)$ into its KT term and its two-children term is exactly equivalent to summing over all ways of extending a suffix set one level deeper.

The Headline Result: CTW is an Exact Bayes Mixture

Theorem 4.5.11 (Bayesian mixture over suffix sets)

$$P_D^{\text{CTW}}(x_{1:n}) = \sum_{S \in \mathcal{C}_D} w_S P_{S, \text{KT}}(x_{1:n}), \quad w_S := 2^{-\Gamma_D(S)}$$

This is the headline result: the CTW root value *is exactly* the Bayes mixture we wanted from Section 4.5, computed in $O(nD)$ time instead of the naive $O(2^{2^D})$ — a double-exponential speedup, and the weights $w_S = 2^{-\Gamma_D(S)}$ form a valid probability distribution over $\mathcal{C}_D$ (Lemma 4.5.10) that favors simpler (more balanced, shallower) suffix sets, exactly as a good Occam's-razor-style prior should.

4.5.3 CTW–PST–KT Redundancy Chain

Convergence properties propagate up the chain $\text{KT} \rightarrow \text{PST-KT} \rightarrow \text{CTW}$:

Theorem 4.5.12 (PST-KT redundancy)

For any $\mathcal{S} \in \mathcal{C}_D$ and parameters $\Theta_{\mathcal{S}}$,

$$\log_2 P_{\mathcal{S}, \Theta_{\mathcal{S}}}(x_{1:n}) - \log_2 P_{\mathcal{S}, \text{KT}}(x_{1:n}) \leq |\mathcal{S}| \gamma(n/|\mathcal{S}|), \quad \gamma(t) := \frac{1}{2} \log_2(t) + 1 \text{ for } t \geq 1.$$

Lemma 4.5.13 (CTW-KT redundancy)

$$\log_2 P_{\mathcal{S}, \text{KT}}(x_{1:n}) - \log_2 P_D^{\text{CTW}}(x_{1:n}) \leq \Gamma_D(\mathcal{S}).$$

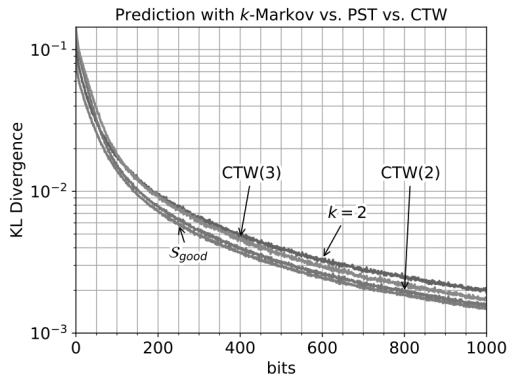
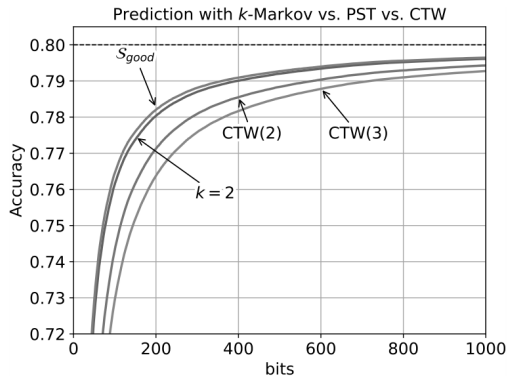
Corollary 4.5.14 (PST-CTW redundancy, combined)

$$\log_2 P_{\mathcal{S}, \Theta_{\mathcal{S}}}(x_{1:n}) - \log_2 P_D^{\text{CTW}}(x_{1:n}) \leq \frac{1}{2} |\mathcal{S}| \log_2 n + \Gamma_D(\mathcal{S}) + |\mathcal{S}|$$

Plain-English meaning: the total extra cost of using CTW instead of knowing the exact true model in advance grows only like $\frac{1}{2} \log_2 n$ per parameter — this matches the *Rissanen lower bound*, the best possible redundancy for *any* estimator, so CTW is asymptotically optimal.

4.5.4 CTW Experiments

Comparing CTW (depths $D = 2, 3$) against the PST with known $\mathcal{S}_{\text{good}}$ and the 2-Markov KT estimator, all on the same environment μ_1 :



CTW Experiments: Discussion

- The PST with known structure is best (it never has to *learn* which contexts matter).
- CTW($D=2$) is close behind, since $D = 2$ already contains $\mathcal{S}_{\text{good}}$ in its model class and quickly concentrates its mixture weight there.
- CTW($D=3$) is slightly slower, since its larger model class ($\mathcal{C}_3 \supset \mathcal{C}_2$) spreads the prior more thinly, giving $\mathcal{S}_{\text{good}}$ a smaller a-priori weight.
- The plain 2-Markov KT estimator is worst — it can never discover the variable-length structure at all, no matter how much data it sees.

Python: A Minimal CTW Implementation (I)

```
class CTWNode:
    def __init__(self):
        self.a = self.b = 0
        self.P_kt = 1.0
        self.P_w = 1.0
        self.children = [None, None]    # [right (bit=0), left (bit=1)]

class CTW:
    def __init__(self, depth):
        self.D = depth
        self.root = CTWNode()

    def _get_or_make(self, node, bit):
        if node.children[bit] is None:
            node.children[bit] = CTWNode()
        return node.children[bit]
```

Each CTWNode stores exactly the four quantities from Definition 4.5.1: the KT counts (a, b) , the KT probability $P_{KT}(a, b)$, and the weighted probability P_w . Nodes are created lazily by `_get_or_make` — this is the online-initialization optimization from Section 4.5.5.

Python: A Minimal CTW Implementation (II)

```
def update(self, context_bits, x):
    """context_bits: most-recent-first list of the D bits preceding x."""
    path = [self.root]
    node = self.root
    for i in range(self.D):
        node = self._get_or_make(node, context_bits[i])
        path.append(node)
    for node in path:                                # update KT stats
        if x == 1:
            node.P_kt *= (node.b + 0.5) / (node.a + node.b + 1); node.b += 1
        else:
            node.P_kt *= (node.a + 0.5) / (node.a + node.b + 1); node.a += 1
    leaf = path[-1]
    leaf.P_w = leaf.P_kt
    for node in reversed(path[:-1]):                 # recompute P_w bottom-up
        left = node.children[1].P_w if node.children[1] else 1.0
        right = node.children[0].P_w if node.children[0] else 1.0
        node.P_w = 0.5 * node.P_kt + 0.5 * left * right

# reproduce Example 4.5.6 on D=2, sequence 1001
ctw = CTW(depth=2)
ctx = [0, 0]                                       # pad with zeros: no history yet
for x in [1, 0, 0, 1]:
    ctw.update(ctx, x)
    ctx = [x] + ctx[:-1]                          # most-recent-first context of length D=2
    print("P_w(root) =", ctw.root.P_w)
# Final output matches P_w(epsilon) = 5/32 = 0.15625 from Figure 4.11(c)
```

4.5.5 Optimizations I

1. Log probabilities. Since $P_{S,KT}(x_{1:n})$ shrinks exponentially in n , ordinary 64-bit floats underflow after only a few hundred bits. The fix: store $\log_2 P_{KT}$ and $\log_2 P_w$ instead, using the numerically stable log-sum operator

$$a \oplus b := \max\{a, b\} + \log_2 \left(1 + 2^{\min\{a, b\} - \max\{a, b\}} \right)$$

which computes $\log_2(2^a + 2^b)$ without ever exponentiating a very negative number. The update rule (4.5.2) becomes

$$\log_2 P_w(s) = \left[\log_2 P_{KT}(a_s, b_s) \oplus (\log_2 P_w(0s) + \log_2 P_w(1s)) \right] - 1 \quad \text{for } \ell(s) < D.$$

4.5.5 Optimizations II

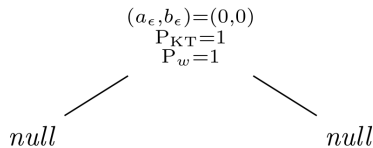
2. Online (lazy) tree initialization. The full context tree $\mathcal{T}_D$ has $O(2^D)$ nodes, intractable to allocate up front for even moderate D . Instead:

- Start with just the root node. Treat all other nodes as `null` (unallocated, implicitly $(a, b) = (0, 0)$, $P_{\text{KT}} = 1$, $P_w = 1$).
- Only ever create a node the first time its context is actually observed (Algorithm 4.6). Since $P_w = 1$ for every null node regardless of depth, formula (4.5.2) simplifies:

$$P_w(s) = \begin{cases} 1 & s \text{ is null} \\ \frac{1}{2}P_{\text{KT}}(a_s, b_s) + \frac{1}{2}P_w(0s)P_w(1s) & \ell(s) < D, s \text{ allocated} \\ P_{\text{KT}}(a_s, b_s) & \ell(s) = D \end{cases}$$

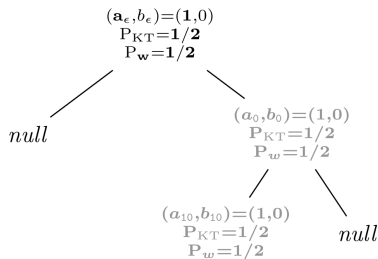
This bounds the number of allocated nodes by $O(nD)$ — in practice far fewer, since many contexts are never seen.

4.5.5 Optimizations III

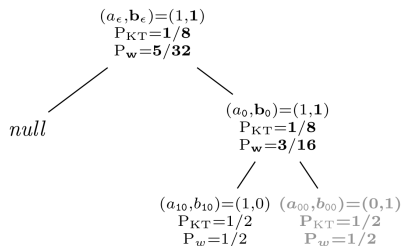


(a) A newly initialized context tree $\mathcal{T}_2$ of depth $D=2$.

4.5.5 Optimizations IV



(b) The context tree $\mathcal{T}_2$ after having processed bit $x_3 = 0$ given context $x_{1:2} = 10$.



(c) The context tree $\mathcal{T}_2$ after having processed bit $x_4 = 1$ given context $x_{1:3} = 100$.

Only the nodes actually visited (black) get allocated; everything else stays *null* and contributes exactly $P_w = 1$ — identical results to the fully pre-allocated tree of Figure 4.11, at a fraction of the memory.

4.5.5 Optimizations V

3. Reverting changes after prediction. Algorithm 4.5 needs $P_D^{\text{CTW}}(1 \mid x_{1:t})$, which requires *tentatively* updating the tree as if $x_{t+1} = 1$, reading off the new root value, and then undoing that update before the real next bit arrives. Rather than copying the whole tree, Algorithm 4.7 simply runs Algorithm 4.6 in reverse: decrement a_s or b_s , and invert the KT-probability update

$$\theta \leftarrow \frac{(a + b + 1)}{b + \frac{1}{2}} \theta \quad (\text{if } x = 1; \text{ symmetric if } x = 0)$$

walking down and back up the same path — an $O(D)$ -time, $O(1)$ -extra-space alternative to duplicating the tree.

Exercises

- C32m Prove Lemma 4.1.5 (the lower bound on $P_{\text{KT}}(a, b)$).
- C10 Find an explicit closed form for $P_{\text{KT}}(n, n)$.
- C10 Construct a countably infinite suffix set that is proper and complete.
- C15 Prove that the CTW update (4.5.2) runs in $O(D)$ time.
- C15 Repeat the redundancy calculation of Example 4.4.1 for the uneven mixture $\hat{\mu}_{w,\alpha} = \alpha\hat{\mu}_1 + (1 - \alpha)\hat{\mu}_2$, and for a general mixture $\hat{\mu}_{w,\theta} = \sum_i \theta_i \hat{\mu}_i$ with $\theta_i \geq 0, \sum_i \theta_i = 1$.
- C30i Implement the CTW algorithm and feed it “random” bits that *you* generate by hand. You may find CTW predicts your next bit correctly more than 60% of the time — humans struggle to act truly randomly!
- C35 Derive a multi-alphabet version of CTW (hint: see [PS99]).
- C32 Suppose a fixed context compressor can reduce the context length by at least half. How does this affect CTW’s redundancy? Implement this compressed-context version of CTW and compare it to the original.

History and References I

- CTW was first briefly introduced in [WST93], with an upper bound on the redundancy. The full explanation and expanded bounds appear in [WST95] — the paper most often cited as “the original CTW paper.” A concise overview by the original authors is given in [WST97].
- The **KT estimator** itself was originally introduced in [KT81], as an efficient, asymptotically-optimal-redundancy estimator for memoryless sources. [SSH12] discusses a *windowed* version of KT for non-stationary environments within CTW.
- **Multi-alphabet extensions:** [TSW93a, TSW93b] extend CTW to arbitrary (non-binary) alphabets; one practical approach is to binarize the alphabet ($\mathcal{A} \hookrightarrow \mathbb{B}^k$ for $k = \lceil \log_2 |\mathcal{A}| \rceil$) [VH18]. The *Sparse Adaptive Dirichlet* (SAD) estimator [Hut13a, VH12] is a near-optimal alternative for large alphabets, later extended to K-distinct reservoir sampling in [Bel15].
- A downside of the original CTW is that the depth D must be fixed in advance; [Wil98] removes this restriction (at increased computational cost). With D infinite or growing logarithmically in n , CTW is asymptotically optimal even on piecewise stationary sources [VCH18].
- Not all sources fit the binary-suffix-tree structure CTW assumes; [WST96] introduces four new model classes with corresponding weighting algorithms. See [Ris83, RST96] for more on prediction suffix trees.

History and References II

- **Beyond prediction:** CTW has been applied to active agents via action-conditional prediction suffix trees [VNHS10] (see Chapter 12). More recently, [MW17] extends CTW to the interactive/partially-observable case, yielding **D2-CTW**, which is less sensitive to aliasing and noise. [PS99] proposes edge-based (rather than node-based) pruning, considering a larger model class.
- **Empirical comparisons:** [BEYY04] compares CTW to other prediction algorithms in practice, finding it outperforms Lempel-Ziv [ZL78, NYEYM03] and Prediction by Partial Match [CW84, RST96].
- **Very recently,** [GDR+23, GMGH+24] trained a Transformer (and LSTMs) on data sampled from CTW, PTW (Section 5.3), and even Solomonoff's distribution. The trained Transformer nearly perfectly mimics CTW and PTW *in-context* — remarkable, since CTW and PTW are Bayes-optimal learners of variable-order Markov / piecewise-i.i.d. processes respectively, meaning the Transformer learned to approximate a Bayes-optimal learner purely from examples, with no further weight updates at test time.
- **Looking ahead (Chapter 5):** further variations include adaptive CTW for non-stationary sources [OHSS12], Partition Tree Weighting (PTW) for piecewise stationary sources [VWBG13], and Context Tree Switching (CTS) [VNHB12].

Summary

- The **KT estimator** predicts i.i.d. Bernoulli sequences with strong, uniformly-bounded redundancy $O(\log n)$, but cannot exploit any temporal structure.
- **k-Markov** models add fixed-length context, but the right k is unknown and costly to over- or under-estimate.
- **Prediction Suffix Trees (PSTs)** generalize to *variable*-length context via suffix sets $\mathcal{S}$, with a natural *model cost* $\Gamma_D(\mathcal{S})$ measuring simplicity.
- **Context Tree Weighting** performs an exact Bayesian mixture over *every* suffix set in $\mathcal{C}_D$ (up to 2^{2^D} of them!) using weights $w_{\mathcal{S}} = 2^{-\Gamma_D(\mathcal{S})}$, updatable in only $O(D)$ time per bit — a genuine double-exponential speedup over the naive approach.
- CTW achieves redundancy matching the **Rissanen lower bound**: it is asymptotically as good as knowing the true model class member in advance.
- CTW is not just theory: it is practical, fast, numerically stabilizable (log-domain), memory-efficient (lazy tree growth), and empirically competitive with state-of-the-art compressors.